

University of Sri Jayewardenepura – Faculty of Applied Sciences
BSc (General) Degree First Year

CSC 1012 – Introduction to Computer Programming **Smart Hospital & Resource Allocation System**

Project Report

Name: Udage Arachchige Yasith Theshan Thathsara

Student ID: AS20250483

GitHub Repository: <https://github.com/theshanthathsara/Smart-Hospital-System>

Table of Contents

- 1. High-Level Architecture
- 2. Data Structure Design
- 3. Core Formulas and Logic
- 4. GitHub Repository
- 5. Assumptions Made During Development

1. High-Level Architecture

This project is a menu-driven Smart Hospital Patient & Resource Allocation System written in the C programming language. The objective was to build a modular system where each part of the hospital's operations — patient intake, bed allocation, billing, reporting, and file handling — is handled by its own dedicated source file, rather than writing all logic inside a single `main()` function.

`main.c` contains only the menu loop. It displays the options to the user, reads their choice, and calls the appropriate function from the relevant module. It does not perform any calculations or data manipulation directly — this keeps `main.c` short and easy to follow, and means each feature can be developed, tested, and debugged independently of the others.

All shared data — lookup tables, the bed occupancy matrix, and every patient's details — is declared in `data.h` and defined in `data.c`. Every other module includes `data.h` so it can read and modify this data. This file effectively acts as the system's in-memory "database."

Module Overview

Module	Responsibility
<code>main.c</code>	Displays the menu, reads user choice, calls other modules
<code>data.h</code> / <code>data.c</code>	Holds all lookup tables and patient parallel arrays
<code>patient.c</code>	Collects and validates new patient registration input
<code>ward.c</code>	Allocates beds and displays ward/bed occupancy
<code>billing.c</code>	Performs all billing calculations and prints the final bill
<code>report.c</code>	Generates summary analytics across all registered patients
<code>fileio.c</code>	Saves/loads bed status and logs patient billing records to disk

Implemented Functions

Function	File	Description
<code>registerPatient()</code>	<code>patient.c</code>	Collects and validates new patient input
<code>displayAllPatients()</code>	<code>patient.c</code>	Lists all registered patients
<code>allocateBed(wardID)</code>	<code>ward.c</code>	Finds and reserves a free bed in the chosen ward
<code>displayBedStatus()</code>	<code>ward.c</code>	Shows bed availability for every ward
<code>calculateWaitTime(i)</code>	<code>billing.c</code>	Calculates estimated waiting time
<code>calculateSurcharge(i)</code>	<code>billing.c</code>	Calculates emergency surcharge
<code>calculateWardCost(i)</code>	<code>billing.c</code>	Calculates ward stay cost
<code>calculateGrossTotal(i)</code>	<code>billing.c</code>	Adds up the gross bill total
<code>calculateDiscount(i)</code>	<code>billing.c</code>	Calculates age-based subsidy discount
<code>calculateFinalAmount(i)</code>	<code>billing.c</code>	Calculates final payable amount
<code>generateBill(i)</code>	<code>billing.c</code>	Runs all billing calculations and prints the bill
<code>generateReports()</code>	<code>report.c</code>	Prints summary analytics for the hospital
<code>saveBedStatus()</code> / <code>loadBedStatus()</code>	<code>fileio.c</code>	Saves/loads bed occupancy between runs
<code>appendPatientRecord(i)</code>	<code>fileio.c</code>	Permanently logs a patient's billing record

2. Data Structure Design

This project uses parallel arrays instead of structs, as explicitly permitted by the assignment instructions. Rather than grouping each patient's fields into a single struct, every field is stored in its own array, and the same index number (i) is used across all of these arrays to represent one patient. For example, index 2 in `patientName[]`, `patientAge[]`, and `urgencyLevel[]` all describe the same patient — the third person registered.

This approach was chosen over structs because it was the method taught and encouraged in the lecture material, and it reinforces core array-handling concepts such as keeping multiple arrays synchronized by index, which is directly relevant to the module's learning outcomes.

Lookup Tables (constant data)

These arrays hold fixed reference data defined by the hospital and do not change while the program runs, aside from the queue counters.

- `specialtyName[4]`, `consultantFee[4]`, `consultantTime[4]`, `patientCap[4]`, `specialtyQueue[4]` — one entry per doctor specialty
- `wardName[4]`, `bedRate[4]`, `bedCapacity[4]` — one entry per hospital ward

Bed Occupancy Matrix

`bedOccupancy[4][20]` is a two-dimensional integer array. The first index selects the ward (0–3), and the second index selects a specific bed within that ward (0–19, although each ward only uses as many as its actual capacity, e.g. ICU only uses indices 0–4). A value of 0 means the bed is available; 1 means it is currently occupied.

Patient Records (parallel arrays, indexed by patient number)

Every patient occupies one index across the following arrays, all sized `MAX_PATIENTS` (100):

- `patientID[]`, `patientName[]`, `patientAge[]`, `emergencyLevel[]` — core patient details
- `patientSpecialty[]`, `isAdmitted[]`, `patientWard[]`, `bedID[]`, `daysAdmitted[]` — specialty and admission details
- `baseFee[]`, `surcharge[]`, `wardCost[]`, `grossTotal[]`, `discount[]`, `finalTotal[]`, `waitingTime[]` — computed billing results

Example: How One Patient's Data Is Spread Across Arrays

The table below shows how a single patient (index 0) is represented — the same index appears in every array, keeping all their information linked together.

Array	Value at index 0
<code>patientID[0]</code>	1001
<code>patientName[0]</code>	"Kamal Perera"
<code>patientAge[0]</code>	70
<code>emergencyLevel[0]</code>	3
<code>patientSpecialty[0]</code>	3 (Cardiology)
<code>isAdmitted[0]</code>	1
<code>patientWard[0]</code>	4 (ICU)
<code>bedID[0]</code>	0 (Bed #01)
<code>daysAdmitted[0]</code>	2
<code>finalTotal[0]</code>	48237.50

Because all arrays are kept in sync by index, sorting patients (for triage priority) requires swapping the same index across every single array at once — this is the main trade-off of parallel arrays compared to structs.

3. Core Formulas and Logic

The billing module implements the following formulas, exactly as specified in the assignment brief.

3.1 Estimated Waiting Time

$$\text{Wait Time} = \text{Current Queue Count for Specialty} \times \text{Avg Time per Patient}$$

The specialty's queue counter is incremented by 1 immediately after a patient registers for that specialty, so the next patient's wait time reflects everyone already ahead of them.

3.2 Emergency Surcharge

0% if Urgency Level = 1 (Normal)

20% if Urgency Level = 2 (Urgent)

50% if Urgency Level = 3 (Critical)

$$\text{Surcharge} = \text{Base Consultation Fee} \times \text{Surcharge Percentage}$$

3.3 Total Ward Stay Cost

$$\text{Total Ward Cost} = \text{Days Admitted} \times \text{Ward Daily Rate}$$

If the patient is not admitted (outpatient), Ward Cost = 0.

3.4 Gross Total Bill

$$\text{Gross Total} = \text{Base Consultation Fee} + \text{Emergency Surcharge} + \text{Total Ward Cost}$$

3.5 Age Subsidy Discount

If Age < 5 OR Age > 65: Discount = Gross Total × 0.15

Otherwise: Discount = 0

3.6 Final Amount Payable

$$\text{Final Payable Amount} = \text{Gross Total} - \text{Discount}$$

3.7 Bed Allocation Logic

When a patient is admitted, `allocateBed(wardID)` searches `bedOccupancy[ward][0]` through `bedOccupancy[ward][capacity-1]` for the first bed marked 0 (available). It marks that bed as 1 (occupied) and returns its index, which is stored in `bedNumber[i]` for that patient. If no free bed is found, the function returns -1, and the patient is registered as an outpatient instead.

3.8 Triage Priority Sorting

Registered patients can be displayed in priority order using a sorting algorithm. Patients with a higher urgencyLevel (3 = Critical) are placed first, followed by Level 2 (Urgent), then Level 1 (Normal). Patients with the same urgency level keep their original registration order. Because parallel arrays are used, every array must be swapped together whenever two patients change position, to keep each patient's data correctly linked by index.

4. GitHub Repository

<https://github.com/theshanthathsara/Smart-Hospital-System>

The repository was maintained as a public repository from the start of the project, with regular, incremental commits spread across the development timeline. A devTheshan branch was used for active development, with completed features merged into main via pull requests once tested and working.

5. Assumptions Made During Development

- If a ward is full when a patient tries to be admitted, they are registered as an outpatient instead, and the user is notified with a message.
- Bed status is saved to beds_status.txt and reloaded automatically the next time the program runs, so bed occupancy persists across sessions.
- Currency values are displayed with 2 decimal places (no thousands-separator commas), for simplicity.
- Patient IDs are generated automatically in sequence, starting from PAT-1001.
- Invalid input (out-of-range numbers, or values outside expected ranges) causes the program to re-prompt the user instead of crashing or exiting.
- patient_records.txt is treated as a permanent, append-only log — records are never deleted or overwritten by the program.
- The system assumes a single user operating the program interactively; no multi-user or concurrent-access handling was implemented, as this was outside the scope of the assignment.